

MeetUp!

System Scenarios & Interaction Flows

v1.0 — Project Initiation

How to Use This Document

Each scenario describes a real user action, traces it through every service it touches, and identifies every event fired. Use this document to validate event contracts, plan integration tests, and understand how services intervene with each other.

Format: trigger → services involved (in order) → events fired → end state.

Scenario 1 — User Signs Up for the First Time

Actor	New user
Entry point	Sign-up page
Meeting type	N/A

Flow

- User enters email on sign-up page and requests OTP
 - Client → API Gateway → Auth Service → Supabase (OTP sent via email)
- User submits OTP
 - Auth Service validates with Supabase → receives user record and JWT
- Auth Service creates user profile in application database
- Auth Service publishes user.registered event to message bus
- Tweening Service consumes user.registered → creates Person node in Neo4j
- Client receives JWT → stored locally → user is logged in

Events fired

Event	Publisher	Consumers
user.registered	Auth Service	Tweening Service

End state

- User record exists in Supabase and application DB
- Person node exists in Neo4j with no relationships yet
- Client holds valid JWT for subsequent requests

Scenario 2 — Meeting Owner Starts a Long-Form Meeting

Actor	Meeting Owner
Entry point	Dashboard — New Meeting
Meeting type	Long-form

Flow

- Meeting Owner clicks New Meeting, selects long-form, adds title and participants
- Client → API Gateway → Meeting Service: POST /meetings
- Meeting Service creates meeting record in PostgreSQL
- Meeting Service requests a Stream video session and Stream chat channel → receives tokens
- Meeting Service publishes meeting.started event
- Context Service consumes meeting.started → creates a new live context document in MongoDB
- Meeting Service returns room details and Stream tokens to client
- Client connects to Stream video and chat directly
- Participants receive meeting invitation notification (via message bus → Notification Consumer)

Events fired

Event	Publisher	Consumers
meeting.started	Meeting Service	Context Service
notification.meeting_invite	Meeting Service	Notification Consumer → participants

End state

- Meeting record in PostgreSQL with status=live
- Live context document open in MongoDB for this meeting_id
- Stream session and chat channel active

Scenario 3 — Participant Joins a Meeting in Progress

Actor	Worker / Group Lead
Entry point	Meeting invite notification
Meeting type	Any

Flow

- Participant taps notification → client requests join token from Meeting Service
- API Gateway validates JWT → Meeting Service confirms participant is invited → issues Stream join token
- Meeting Service logs participant join in PostgreSQL
- Meeting Service publishes participant.joined event
- Context Service consumes participant.joined → appends participant to context document's participant_ids
- Client connects to Stream session — participant appears in the call

Events fired

Event	Publisher	Consumers
participant.joined	Meeting Service	Context Service

End state

- Participant visible in meeting room
- Context document participant list updated

Scenario 4 — Meeting Owner Creates a Task During a Meeting

Actor	Meeting Owner
Entry point	Task panel inside active meeting
Meeting type	Long-form

Flow

- Meeting Owner opens task panel, fills in title and assigns to two participants
- Client → API Gateway → Task Service: POST /tasks with meeting_id and assignee_ids
- Task Service saves task in PostgreSQL with state=backlog and created_in_meeting_id set
- Task Service publishes task.created event
- Task Service publishes task.assigned event for each assignee
- Tweening Service consumes task.created → queries Neo4j membership resolver for member suggestions → returns to dashboard
- Tweening Service consumes task.assigned → creates Group node in Neo4j (state=forming) → creates WORKS_ON relationship to Task
- Meeting Service consumes group.formed → spawns a sub-meeting (private breakout room) for the group
- Each assignee receives a group invitation notification (message bus → Notification Consumer)
- Context Service consumes task.created → appends task_id to current meeting's tasks_ref
- AI Service consumes task.created → runs assignee suggester via Neo4j → pushes suggestions to dashboard

Events fired

Event	Publisher	Consumers
task.created	Task Service	Tweening Service, Context Service, AI Service
task.assigned	Task Service	Tweening Service, Meeting Service, Notification Consumer
group.formed	Tweening Service	Meeting Service, Context Service
submeeting.spawned	Meeting Service	Context Service

End state

- Task record in PostgreSQL (state=backlog)
- Group node in Neo4j (state=forming) with WORKS_ON → Task
- Sub-meeting created and waiting for assignees to join
- Assignees notified with invitation

Scenario 5 — Assignee Accepts Task Invitation and Joins Group

Actor	Worker
Entry point	Group invitation notification
Meeting type	Sub-meeting

Flow

- Worker taps invitation notification → sees task summary, group details
- Worker clicks Accept → Client → Tweening Service: POST /groups/{id}/members
- Tweening Service appends MEMBER_OF relationship in Neo4j with joined_at timestamp
- Tweening Service publishes member.joined event
- If this is the second member joining: Tweening Service transitions group state to active
- Meeting Service consumes member.joined → issues Stream sub-meeting join token to the new member
- Task Service transitions task state from backlog → in_progress (triggered by group reaching active state)
- Task Service publishes task.updated event
- Context Service logs group join as an action in the meeting context document
- Worker's client connects to the sub-meeting Stream session — video and chat now available

Events fired

Event	Publisher	Consumers
member.joined	Tweening Service	Meeting Service, Context Service
task.updated	Task Service	Context Service, Dashboard

End state

- MEMBER_OF relationship in Neo4j with timestamp
- Group state=active in Neo4j
- Task state=in_progress in PostgreSQL
- Worker connected to sub-meeting video and group chat

Scenario 6 — Meeting Ends and AI Processing Runs

Actor	Meeting Owner
Entry point	End Meeting button
Meeting type	Long-form

Flow

- Meeting Owner clicks End Meeting
- Meeting Service closes the Stream session and marks meeting as ended in PostgreSQL
- Meeting Service publishes meeting.ended event
- Context Service consumes meeting.ended → finalises transcript assembly, marks context document status=complete
- AI Service consumes meeting.ended → runs meeting summariser: fetches full transcript from Context Service, sends structured prompt to LLM, receives JSON with summary, decisions, open questions
- AI Service writes summary back to Context Service: PATCH /context/{meeting_id}/summary
- Context Service updates MongoDB document with the AI summary
- AI Service concurrently runs task extractor: identifies action items in transcript, creates draft tasks in Task Service
- Task Service saves draft tasks (status=draft) and publishes ai.tasks.extracted event
- Dashboard displays draft tasks — participant sees 'Confirm or dismiss' prompt
- Notification Consumer sends post-meeting summary notification to all participants

Events fired

Event	Publisher	Consumers
meeting.ended	Meeting Service	Context Service, AI Service, Tweening Service
ai.summary.ready	AI Service	Context Service
ai.tasks.extracted	AI Service	Task Service, Dashboard
notification.meeting_summary	Notification Consumer	All participants

End state

- Meeting record in PostgreSQL with status=ended
- Context document in MongoDB: complete, with transcript, decisions, topics, and AI summary
- Draft tasks visible in dashboard awaiting confirmation
- All participants notified with summary

Scenario 7 — User Opens a New Meeting With Prior History

Actor	Any participant
Entry point	New meeting with recurring participants
Meeting type	Long-form

Flow

- Meeting Owner creates new meeting and adds participants who were in previous meetings together
- Client requests pre-meeting briefing: GET /context/briefing?meetingId=X
- Context Service briefing builder queries MongoDB for prior sessions with matching participant_ids or tween_group_ids
- Briefing builder fetches open tasks from Task Service: GET /tasks?pending&participantIds=...
- Briefing builder composes response: last session summary, decisions made, open tasks
- Client renders briefing panel before the call begins — participants can review before joining

Events fired

None — this is a synchronous read-only flow.

End state

- Participants see: what was discussed last time, decisions made, tasks still open
- Meeting starts with shared context — no re-explaining needed

Scenario 8 — Task Is Marked Done and Group Dissolves

Actor	Group Lead or Meeting Owner
Entry point	Task board
Meeting type	N/A (async)

Flow

- Group Lead marks task as complete: PATCH /tasks/{id}/state with state=done
- Task Service runs state machine validation (checks no open subtasks, no blocking dependencies)
- Task Service updates state in PostgreSQL and publishes task.completed event
- Tweening Service consumes task.completed → transitions group state to dissolved
- Tweening Service writes COLLABORATED_WITH relationship in Neo4j for each pair of group members — weighted by collaboration duration
- Tweening Service publishes group.dissolved event
- Context Service consumes group.dissolved → appends dissolution to relevant context documents
- Meeting Service consumes group.dissolved → closes the sub-meeting if still active
- AI Service consumes group.dissolved → uses completed task as signal for future assignee suggestions (weight update)
- All group members notified that task is complete and group has dissolved (Notification Consumer)

Events fired

Event	Publisher	Consumers
task.completed	Task Service	Tweening Service, Context Service, AI Service
group.dissolved	Tweening Service	Context Service, Meeting Service, AI Service
notification.task_complete	Notification Consumer	Group members

End state

- Task state=done in PostgreSQL
- Group state=dissolved in Neo4j
- COLLABORATED_WITH edges written in Neo4j — graph updated for future suggestions
- Sub-meeting closed if it was still active

Scenario 9 — Group Lead Transfers Leadership

Actor	Group Lead
Entry point	Group settings panel
Meeting type	N/A

Flow

- Group Lead selects a group member and transfers lead role
- Client → Tweening Service: PATCH /groups/{id}/lead with new_lead_id
- Tweening Service ends the current LEADS relationship (adds to timestamp) in Neo4j
- Tweening Service creates new LEADS relationship for the new lead in Neo4j
- Tweening Service updates role_in_group on both parties' MEMBER_OF relationships
- Tweening Service publishes group.evolved event
- Context Service consumes group.evolved → logs role change as an action in any active meeting context
- New Group Lead receives notification of role assignment

Events fired

Event	Publisher	Consumers
group.evolved	Tweening Service	Context Service, Dashboard
notification.role_changed	Notification Consumer	New Group Lead

End state

- Neo4j: previous LEADS relationship ended, new one created with timestamp
- Role history fully preserved in the graph — auditable

Scenario 10 — Meeting Owner Spawns a Sub-Meeting From Main Meeting

Actor	Meeting Owner
Entry point	Inside an active long-form meeting
Meeting type	Sub-meeting spawned from long-form

Flow

- Meeting Owner selects a tween group from the participant panel and clicks 'Spawn sub-meeting'
- Client → Meeting Service: POST /meetings/{parentId}/submeeting with group_id
- Meeting Service creates a new meeting record in PostgreSQL linked to the parent via parent_meeting_id
- Meeting Service requests a new Stream session and private chat channel scoped to the group
- Meeting Service publishes submeeting.spawned event
- Context Service consumes submeeting.spawned → creates a child context document linked to the parent meeting's context
- Tweening Service consumes submeeting.spawned → appends FORMED_IN relationship from Group to the new Meeting node in Neo4j
- Group members receive notification that their sub-meeting is ready to join
- Members can now join the sub-meeting without leaving the main meeting stream

Events fired

Event	Publisher	Consumers
submeeting.spawned	Meeting Service	Context Service, Tweening Service
notification.submeeting_ready	Notification Consumer	Group members

End state

- Child meeting record in PostgreSQL linked to parent
- Child context document in MongoDB linked to parent context
- FORMED_IN relationship written in Neo4j
- Group members can join the private sub-meeting video and chat

Scenario 11 — AI Draft Task Is Confirmed by Participant

Actor	Worker or Meeting Owner
Entry point	Post-meeting dashboard prompt
Meeting type	Post long-form meeting

Flow

- Participant sees 'We found 2 possible tasks' prompt in dashboard
- Participant reviews each draft task: title, description, suggested assignees
- For each confirmed task: Client → Task Service: PATCH /tasks/{id}/state with state=backlog
- Task Service updates status from draft → backlog and publishes task.created event (now official)
- For each dismissed task: Client → Task Service: DELETE /tasks/{id}
- On confirmation: Tweening Service reacts to task.created — begins group suggestion flow (Scenario 4)
- Confirmed tasks appear on the project task board

Events fired

Event	Publisher	Consumers
task.created (confirmed)	Task Service	Tweening Service, Context Service, AI Service

End state

- Confirmed tasks are live records in PostgreSQL — full lifecycle begins
- Dismissed tasks are deleted

Event Reference — All System Events

Event	Publisher	Primary consumers
user.registered	Auth Service	Tweening Service
meeting.started	Meeting Service	Context Service
meeting.ended	Meeting Service	Context Service, AI Service, Tweening Service
participant.joined	Meeting Service	Context Service
participant.left	Meeting Service	Context Service
submeeting.spawned	Meeting Service	Context Service, Tweening Service
transcript.chunk	Meeting Service	Context Service, AI Service
task.created	Task Service	Tweening Service, Context Service, AI Service
task.assigned	Task Service	Tweening Service, Meeting Service
task.updated	Task Service	Context Service, Dashboard
task.completed	Task Service	Tweening Service, Context Service, AI Service
group.formed	Tweening Service	Meeting Service, Context Service, Task Service
group.evolved	Tweening Service	Context Service, Dashboard
group.dissolved	Tweening Service	Context Service, Meeting Service, AI Service
member.joined	Tweening Service	Meeting Service, Context Service
member.left	Tweening Service	Context Service
ai.summary.ready	AI Service	Context Service
ai.tasks.extracted	AI Service	Task Service, Dashboard